D15A - 04

ML-DL Experiment 7

Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

Your dataset (Churn_Modelling.csv) is typically derived from banking/customer datasets used for predicting **customer churn**.

Source (typical reference):

- Kaggle: *Churn Modelling Dataset*
- Contains customer details from a bank

Dataset link: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

Feature	Description
RowNumber	Index
CustomerId	Unique ID
Surname	Customer name
CreditScore	Credit score
Geography	Country (France, Spain, Germany)
Gender	Male/Female
Age	Customer age
Tenure	Years with bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Credit card (0/1)
IsActiveMember	Active or not
EstimatedSalary	Salary

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of layers of neurons.

Neuron Computation:

Each neuron computes:

$$Z = \sum(w_i x_i) + b$$

$$A = f(Z)$$

Where:

- wi = weights
- xi = inputs
- b = bias
- f(Z) = activation function

Activation Functions:

- ReLU:

$$f(x) = \max(0, x)$$

- Sigmoid (used in output layer):

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Classification):

$$Loss = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Backpropagation:

- Computes gradients using chain rule
- Updates weights using:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where:

- η = learning rate

4. Algorithm Limitations

- Requires large dataset
- Black-box model (low interpretability)
- Overfitting risk
- Computationally expensive
- Needs hyperparameter tuning

5. Methodology / Workflow (Google Colab)

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import tensorflow as tf
```

Step 2: Load Dataset

```
df = pd.read_csv('/content/Churn_Modelling.csv')
df.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1

Step 3: Data Preprocessing

Remove unnecessary columns

```
df = df.drop(['RowNumber','CustomerId','Surname'], axis=1)
df = pd.get_dummies(df, drop_first=True)
```

Encode categorical data

Step 4: Split Data

```
from sklearn.model_selection import train_test_split

X = df.drop('Exited', axis=1)
y = df['Exited']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Step 5: Feature Scaling

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Step 6: Build ANN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))
```

Step 7: Compile Model

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Step 8: Train Model

```
model.fit(X_train, y_train, batch_size=32, epochs=50)

250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8602 - loss: 0.3402
Epoch 23/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8622 - loss: 0.3394
Epoch 24/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8615 - loss: 0.3389
Epoch 25/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8631 - loss: 0.3383
Epoch 26/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8622 - loss: 0.3377
Epoch 27/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8637 - loss: 0.3369
Epoch 28/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8625 - loss: 0.3365
Epoch 29/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8630 - loss: 0.3362
Epoch 30/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8635 - loss: 0.3356
Epoch 31/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8636 - loss: 0.3352
Epoch 32/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8641 - loss: 0.3348
Epoch 33/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8644 - loss: 0.3345
Epoch 34/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8639 - loss: 0.3340
Epoch 35/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8640 - loss: 0.3338
```

Step 9: Predictions

```
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

Step 10: Evaluation

```
from sklearn.metrics import confusion_matrix, accuracy_score

print(confusion_matrix(y_test, y_pred))
print(accuracy_score(y_test, y_pred))
```

```
[[1514   74]
 [ 230  182]]
0.848
```

Workflow Representation Dataset → Preprocessing → Encoding → Scaling → ANN Model → Training → Prediction → Evaluation

6. Performance

Analysis Metrics:

- Accuracy
- Confusion Matrix
- Precision, Recall, F1-score

Example:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.87      0.95      0.91      1588
           1       0.71      0.44      0.54       412

    accuracy                           0.85      2000
   macro avg       0.79      0.70      0.73      2000
weighted avg       0.84      0.85      0.83      2000
```

Interpretation:

- High accuracy → good prediction
- Recall important → catching churn customers

Parameter	Effect
Epochs	Training duration
Batch size	Learning stability
Learning rate	Speed of convergence
Hidden layers	Model complexity
Neurons	Capacity

Conclusion

- ANN successfully predicts customer churn
- Handles nonlinear relationships effectively
- Performance improves with tuning and preprocessing
- Best suited for large structured dataset